

Thesis: A Comparative Study of Large Language Models for Financial Sentiment Analysis and Their Predictive Potential for Short-Term Stock Price Movement

Component: Experiment 2 - Dataset Preparation

Description: This notebook filters and samples the FNSPID dataset (Dong et al., 2024) for 10 selected companies across the period January 2018 to June 2020 (last update to the dataset). It uses FinBERT to remove neutral headlines, randomly samples 100 headlines per company (1000 total headlines), retrieves next-day forward stock price movement from Yahoo Finance, merges both datasets and saves the final dataset as a CSV file to evaluate the four LLM models on it.

Select T4 GPU as runtime.

```
# Install all required libraries – Dataset Preparation
```

```
!pip install -q transformers datasets torch
!pip install -q yfinance pandas matplotlib seaborn
```

```
print("All libraries installed successfully.")
```

```
All libraries installed successfully.
```

```
# Import required libraries for dataset preparation
```

```
import pandas as pd
import numpy as np
import torch
import warnings
```

```
import yfinance as yf
import matplotlib.pyplot as plt
import seaborn as sns
from transformers import pipeline
```

```
warnings.filterwarnings("ignore")
```

```
# Check GPU
```

```
print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU detected : {torch.cuda.get_device_name(0)}")
else:
    print("No GPU detected")
```

```
PyTorch version : 2.10.0+cu128
CUDA available : True
GPU detected : Tesla T4
```

Add the API Key from Hugging Face using "Add New Secret" in Google Colab

```
# Connect to Hugging Face
from google.colab import userdata
from huggingface_hub import login
```

```
hf_token = userdata.get("HF_TOKEN")
login(token=hf_token)
```

```
print("Hugging Face login successful.")
```

```
Hugging Face login successful.
```

```
# Mount Google Drive and download FNSPID dataset
# Source: Dong et al. (2024)
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
from huggingface_hub import hf_hub_download
import os
```

```
print("Downloading FNSPID dataset...")
```

```
file_path_all = hf_hub_download(
    repo_id="Zihan1004/FNSPID",
    filename="Stock_news/All_external.csv",
    repo_type="dataset",
    token=hf_token)
```

```
)

size_gb = os.path.getsize(file_path_all) / (1024**3)
print(f"Downloaded successfully.")
print(f"File size: {size_gb:.2f} GB")
```

Mounted at /content/drive
Downloading FNSPID dataset...

Stock_news/All_external.csv: 100%

5.73G/5.73G [00:37<00:00, 166MB/s]

Downloaded successfully.
File size: 5.34 GB

```
# Filter FNSPID for 10 tickers and date range 2018-2020

TICKERS_FINAL = ["AAPL", "AMZN", "GOOGL", "INTC", "JPM",
                 "NFLX", "NVDA", "TSLA", "ORCL", "FB"]

chunks_final = []
total_read = 0

print("Filtering FNSPID for 10 tickers and 2018-2020...")

for chunk in pd.read_csv(
    file_path_all,
    chunksize=50000,
    encoding="latin-1",
    on_bad_lines="skip",
    engine="python"
):
    filtered = chunk[chunk["Stock_symbol"].isin(TICKERS_FINAL)].copy()

    if len(filtered) > 0:
        filtered["Date"] = pd.to_datetime(
            filtered["Date"], errors="coerce", utc=True
        ).dt.tz_localize(None)

        filtered = filtered[
            (filtered["Date"] >= "2018-01-01") &
            (filtered["Date"] <= "2020-06-30")
        ]

        if len(filtered) > 0:
            chunks_final.append(
                filtered[["Stock_symbol", "Date", "Article_title"]]
            )

    total_read += len(chunk)
    if total_read % 2000000 == 0:
        print(f" Processed {total_read:,} rows...")

df_fnspid_filtered = pd.concat(chunks_final).reset_index(drop=True)

print(f"\nDone.")
print(f"Total rows : {len(df_fnspid_filtered):,}")
print(f"\nPer ticker counts:")
print(df_fnspid_filtered["Stock_symbol"].value_counts())
```

```
Filtering FNSPID for 10 tickers and 2018-2020...
Processed 2,000,000 rows...
Processed 4,000,000 rows...
Processed 6,000,000 rows...
Processed 8,000,000 rows...
Processed 10,000,000 rows...
Processed 12,000,000 rows...
```

Done.
Total rows : 13,987

Per ticker counts:

Stock_symbol	
JPM	2883
INTC	2475
NFLX	1974
TSLA	1875
GOOGL	1754
NVDA	1343
ORCL	499
AAPL	473
FB	433
AMZN	278

Name: count, dtype: int64

```
# Load FinBERT and classify all headlines
# Keep only positive and negative headlines – exclude neutral headlines
```

```
print("Loading FinBERT for pre-filtering...")
```

```
finbert = pipeline(
    "text-classification",
    model="ProsusAI/finbert",
    tokenizer="ProsusAI/finbert",
    device=0 if torch.cuda.is_available() else -1
)
```

```
print("FinBERT loaded. Running on all headlines...")
```

```
headlines = df_fnspid_filtered["Article_title"].tolist()
```

```
finbert_results = finbert(headlines, batch_size=32)
df_fnspid_filtered["finbert_sentiment"] = [r["label"].lower() for r in finbert_results]
```

```
# Keep only positive and negative
df_fnspid_sentiment = df_fnspid_filtered[
    df_fnspid_filtered["finbert_sentiment"] != "neutral"
].copy()
```

```
print(f"\nTotal headlines before filter : {len(df_fnspid_filtered):,}")
print(f"After removing neutral      : {len(df_fnspid_sentiment):,}")
print(f"Neutral removed              : {len(df_fnspid_filtered) - len(df_fnspid_sentiment):,}")
print(f"\nSentiment distribution:")
print(df_fnspid_sentiment["finbert_sentiment"].value_counts())
print(f"\nPer ticker after filter:")
print(df_fnspid_sentiment["Stock_symbol"].value_counts())
```

Loading FinBERT for pre-filtering...

config.json: 100% 758/758 [00:00<00:00, 79.9kB/s]

pytorch_model.bin: 100% 438M/438M [00:03<00:00, 277MB/s]

Loading weights: 100% 201/201 [00:00<00:00, 739.30it/s, Materializing param=classifier.weight]

BertForSequenceClassification LOAD REPORT from: ProsusAI/finbert

Key	Status	
bert.embeddings.position_ids	UNEXPECTED	

Notes:

– UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch

tokenizer_config.json: 100% 252/252 [00:00<00:00, 12.5kB/s]

model.safetensors: 100% 438M/438M [00:03<00:00, 210MB/s]

vocab.txt: 232k/? [00:00<00:00, 13.9MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 4.30kB/s]

FinBERT loaded. Running on all headlines...

```
Total headlines before filter : 13,987
After removing neutral      : 6,052
Neutral removed              : 7,935
```

Sentiment distribution:

```
finbert_sentiment
positive    3273
negative    2779
Name: count, dtype: int64
```

Per ticker after filter:

```
Stock_symbol
INTC      961
JPM       939
TSLA      923
NFLX      906
NVDA      792
GOOGL     712
ORCL      259
AAPL      247
FB        190
AMZN      123
Name: count, dtype: int64
```

```
# Download stock price data and compute next-day movement labels
```

```
print("Downloading stock price data...")
```

```
tickers_without_fb = ["AAPL", "AMZN", "GOOGL", "INTC", "JPM",
                      "NFLX", "NVDA", "ORCL", "TSLA"]
```

```

price_main = yf.download(
    tickers_without_fb,
    start="2018-01-01",
    end="2020-07-01",
    progress=False
)["Close"]

# FB/Meta separately
price_fb = yf.download(
    "META",
    start="2018-01-01",
    end="2020-07-01",
    progress=False
)["Close"]

price_fb.name = "FB"

# Combine
price_data = price_main.copy()
price_data["FB"] = price_fb

# Reshape to long format
price_long = price_data.reset_index().melt(
    id_vars="Date",
    var_name="ticker",
    value_name="close_price"
)

price_long["date"] = pd.to_datetime(price_long["Date"]).dt.date
price_long = price_long.drop(columns="Date")
price_long = price_long.sort_values(["ticker", "date"]).reset_index(drop=True)

# Next-day movement label (1 trading day forward)
price_long["next_close_1d"] = price_long.groupby("ticker")["close_price"].shift(-1)
price_long["movement"] = (price_long["next_close_1d"] > price_long["close_price"]).astype(int)
price_long = price_long.dropna(subset=["next_close_1d"])

print(f"Price data downloaded.")
print(f"Total rows      : {len(price_long):,}")
print(f"Tickers          : {price_long['ticker'].nunique()}")
print(f"\nNext-day movement distribution:")
print(price_long["movement"].value_counts())

```

```

Downloading stock price data...
Price data downloaded.
Total rows      : 6,270
Tickers         : 10

```

```

Next-day movement distribution:
movement
1      3325
0      2945
Name: count, dtype: int64

```

```

# Sample 100 sentiment-bearing headlines per company
# From trading days only, then merge with next-day price movement

trading_days = set(price_long["date"].unique())

# Filter to trading days only
df_fnspid_trading = df_fnspid_sentiment[
    df_fnspid_sentiment["Date"].dt.date.isin(trading_days)
].copy()

print("Sentiment-bearing headlines on trading days per ticker:")
print(df_fnspid_trading["Stock_symbol"].value_counts())
print(f"\nTotal available: {len(df_fnspid_trading):,}")

# Sample 100 per company
SAMPLES_PER_COMPANY = 100
sampled_parts = []

for ticker in TICKERS_FINAL:
    df_ticker = df_fnspid_trading[
        df_fnspid_trading["Stock_symbol"] == ticker
    ]
    n = min(SAMPLES_PER_COMPANY, len(df_ticker))
    sampled = df_ticker.sample(n=n, random_state=42)
    sampled_parts.append(sampled)

df_sample = pd.concat(sampled_parts).reset_index(drop=True)
df_sample = df_sample[["Stock_symbol", "Date", "Article_title", "finbert_sentiment"]].copy()

```

```

df_sample.columns = ["ticker", "date", "headline", "finbert_sentiment"]
df_sample["date"] = pd.to_datetime(df_sample["date"]).dt.date

# Merge with price data
df_merged = df_sample.merge(
    price_long[["ticker", "date", "close_price", "next_close_1d", "movement"]],
    on=["ticker", "date"],
    how="inner"
)

print("\n" + "=" * 45)
print("Final Dataset Statistics")
print("=" * 45)
print(f"Headlines sampled : {len(df_sample):,}")
print(f"Matched with prices : {len(df_merged):,}")
print(f"Unmatched : {len(df_sample) - len(df_merged):,}")
print(f"\nPer ticker:")
print(df_merged["ticker"].value_counts().sort_index())
print(f"\nNext-day movement distribution:")
print(df_merged["movement"].value_counts())
print(f"\nSentiment distribution:")
print(df_merged["finbert_sentiment"].value_counts())

```

Sentiment-bearing headlines on trading days per ticker:

```

Stock_symbol
INTC      916
TSLA     900
NFLX     893
JPM       890
NVDA     787
GOOGL    696
ORCL     256
AAPL     235
FB        184
AMZN     119
Name: count, dtype: int64

```

Total available: 5,876

Final Dataset Statistics

```

Headlines sampled : 1,000
Matched with prices : 1,000
Unmatched : 0

```

Per ticker:

```

ticker
AAPL      100
AMZN      100
FB         100
GOOGL     100
INTC       100
JPM        100
NFLX       100
NVDA       100
ORCL       100
TSLA       100
Name: count, dtype: int64

```

Next-day movement distribution:

```

movement
1      508
0      492
Name: count, dtype: int64

```

Sentiment distribution:

```

finbert_sentiment
positive      514
negative     486
Name: count, dtype: int64

```

Save final dataset to Google Drive for evaluating four LLM models

```

import os
os.makedirs("/content/drive/MyDrive/Thesis_Data", exist_ok=True)

df_merged.to_csv("/content/drive/MyDrive/Thesis_Data/exp2_dataset.csv", index=False)

print("Saved to Google Drive - exp2_dataset.csv")
print(f"\nColumns: {list(df_merged.columns)}")
print(f"\nSample rows:")
print(df_merged.head(3))

```

Saved to Google Drive - exp2_dataset.csv

Columns: ['ticker', 'date', 'headline', 'finbert_sentiment', 'close_price', 'next_close_1d', 'movement']

Sample rows:

	ticker	date	headline \
0	AAPL	2020-04-30	Apple CEO Tim Cook: Store Traffic Not Back To ...
1	AAPL	2020-03-13	Apple Gets Upgrade, Futures Reverse Downward C...
2	AAPL	2020-03-24	Rise In Semiconductor, Cloud Computing Stocks ...

	finbert_sentiment	close_price	next_close_1d	movement
0	negative	70.989700	69.846794	0
1	negative	67.164764	58.524212	0
2	positive	59.652592	59.323994	0